PROJECT REPORT

Student Performance Predictor

Using Machine Learning (Linear Regression)

Name	Govind Parashuram Jadhav
Registration No.	25BAI11219
Project Type	AI/ML Beginner Project
Technology	Python, Linear Regression
Date	30 March 2026

1. Introduction and Problem Statement

1.1 Background

Education is one of the most important pillars of society, and student performance is a key concern for educators, parents, and policymakers alike. Understanding how study habits directly influence academic outcomes can help students make better decisions about how to allocate their time and effort.

In many academic institutions, performance is still evaluated purely after the fact — once examinations are over and results are published. There is little effort made to predict or anticipate how a student might perform based on measurable inputs like the number of hours they dedicate to studying. This reactive approach leaves little room for early intervention.

1.2 The Problem

The core problem addressed by this project is straightforward yet significant: Can we accurately predict a student's examination marks based solely on the number of hours they have studied? And further, can we classify that predicted performance into a binary outcome — PASS or FAIL — to give students and educators a clear, actionable result?

Without a predictive tool, students often have no concrete way of knowing whether the time they are investing in studies is sufficient. They may study for too few hours and be unaware of the risk of failure, or they may over-invest without understanding diminishing returns.

1.3 Why This Problem Matters

Predicting student performance has practical significance across multiple stakeholders:

- Students can self-assess their preparation and adjust study time accordingly.
- Educators can identify at-risk students early and offer additional support before it is too late.
- Parents can monitor their child's academic trajectory and intervene proactively.
- Institutions can make data-driven decisions to improve teaching outcomes and pass rates.

Even a simple, lightweight prediction model — as built in this project — demonstrates how data and machine learning can be applied to solve real-world educational challenges without requiring complex infrastructure or large datasets.

2. Approach to Solving the Problem

2.1 Methodology Overview

The project follows a supervised machine learning workflow. The general approach consists of four phases: data collection, model training, prediction, and result classification. The entire solution is implemented from scratch in Python without relying on external machine learning libraries such as scikit-learn, which demonstrates a deeper understanding of the underlying algorithm.

2.2 Dataset

A small, hand-curated dataset was used for this project. The dataset is stored in a CSV file (data.csv) and contains 9 records. Each record captures three fields:

Column	Sample Values	Description
study_hours	2, 3, 4 ... 10	Number of hours the student studied (independent variable X)
attendance	60, 65, 70 ... 95	Attendance percentage (present in dataset but not used in regression)
marks	45, 50, 55 ... 85	Marks obtained by the student (dependent variable Y)

The dataset is deliberately small and linear, making it a suitable starting point for a first machine learning project while still allowing meaningful pattern recognition.

2.3 Algorithm — Simple Linear Regression

Linear Regression is a supervised learning algorithm that models the relationship between an independent variable (X) and a dependent variable (Y) by fitting a straight line through the data points. The mathematical equation governing this relationship is:

$$y = mx + c$$

Where y is the predicted marks, x is the study hours, m is the slope (rate of change of marks per hour studied), and c is the y-intercept (base mark when study hours = 0).

The slope (m) and intercept (c) are calculated using the Ordinary Least Squares (OLS) method, which minimizes the sum of squared differences between actual and predicted values:

$$m = \Sigma[(x_i - \bar{x})(y_i - \bar{y})] \;/\; \Sigma[(x_i - \bar{x})^2]$$

$$c = \bar{y} - m \times \bar{x}$$

2.4 Classification Logic

Once the predicted mark is obtained, a simple threshold-based classification is applied to determine the student's result:

Predicted Marks	Result
Marks ≥ 40	**PASS**
Marks < 40	**FAIL**

3. Key Design Decisions

3.1 Building the Algorithm from Scratch

One of the most deliberate decisions made in this project was to implement the Linear Regression algorithm manually, rather than importing it from a library like scikit-learn. This was intentional and educationally motivated.

Writing the OLS formulae manually — computing means, iterating over data points, calculating the slope and intercept — ensures that the developer truly understands what is happening mathematically inside the model, rather than treating it as a black box. This is especially important for a first AI/ML project where the primary goal is learning the concept, not maximizing engineering speed.

3.2 Separating Model Logic from Application Logic

The project is organized into two Python files: model.py and main.py. This architectural decision follows the principle of separation of concerns:

- model.py contains the LinearRegressionModel class and the data loading function — the core ML logic.
- main.py contains the user interaction, input handling, and output formatting — the application logic.

This separation makes the codebase cleaner, easier to maintain, and easier to test. It also mimics real-world software engineering practices and demonstrates good coding habits from the beginning.

3.3 Using a CSV File for Data Storage

Rather than hardcoding data directly into the script, the project loads the dataset from an external CSV file (data.csv). This makes the model flexible — the dataset can be updated or replaced without modifying any Python code, which is a fundamental principle of data-driven software design.

3.4 Using Only Study Hours for Prediction

While the dataset includes an attendance column, the model was built to use only study_hours as the independent variable. This was a conscious simplification to keep the project focused on demonstrating simple linear regression (one input, one output). Using two input variables would have required multiple linear regression, which is a more complex extension appropriate for a follow-up project.

3.5 Threshold for Pass/Fail Classification

A threshold of 40 marks was chosen as the pass/fail boundary. This is a commonly used passing criterion in Indian educational systems (typically 40% of 100), making the project's logic realistic and contextually appropriate.

4. Challenges Faced

4.1 Understanding the Mathematical Foundation

The first and most significant challenge was grasping the mathematics behind linear regression. Concepts such as the mean, deviation, sum of squared differences, and how these combine to produce the optimal slope and intercept are not immediately intuitive for someone encountering them for the first time.

Translating the textbook formula into actual working Python code — without the help of a library — required careful study, step-by-step verification, and testing with manually calculated values to confirm correctness.

4.2 File Path Management

A practical challenge encountered during development was managing relative file paths between the Source code folder and the data folder. Since main.py is run from inside the Source code directory, the path to data.csv had to use ../ (parent directory navigation). Getting this path right required understanding of how Python resolves relative paths using os.path.join.

4.3 Input Validation

Handling invalid user input was another area that required attention. If a user types a non-numeric value when prompted for study hours, the program would crash with a ValueError. This was addressed using a try-except block in main.py to catch the error and display a friendly message instead of an unhandled exception.

4.4 Dataset Limitations

The dataset used is extremely small (9 records) and perfectly linear, meaning all data points fall almost exactly on a straight line. While this is ideal for learning, it is not representative of real-world data, which is noisy and contains outliers. A real student performance dataset would have far more variability, and a simple linear regression might not capture the full complexity.

This was acknowledged as a limitation rather than a problem to be solved within the current scope of the project.

5. Key Learnings

5.1 Technical Learnings

Machine Learning Fundamentals

- Gained a practical understanding of what supervised learning means — training a model on labelled data so it can predict outcomes for unseen inputs.
- Understood the intuition behind the OLS method: that the best-fit line is the one that minimizes the total prediction error across all training samples.
- Learned how the slope (m) captures the rate of change — in this case, how many marks a student can expect to gain for each additional hour of study.

Python Programming

- Applied object-oriented programming (OOP) by encapsulating the model logic in a Python class.
- Practiced reading structured data from a CSV file using the csv module's DictReader.
- Implemented error handling using try-except blocks for both file I/O and user input.
- Understood how to structure a multi-file Python project with imports between modules.

Project Organization

- Learned the importance of separating concerns between model logic and application logic.
- Understood how to organize a project with clear folders (Source code, data) and meaningful filenames.
- Used Git and GitHub for version control, which is a foundational skill for all software development.

5.2 Conceptual Learnings

Beyond the technical skills, this project reinforced several important conceptual insights:

- Data quality matters: A model is only as good as the data it is trained on. A clean, well-structured dataset, even if small, produces a reliable model.
- Simplicity is powerful: A single-variable linear regression, despite being one of the simplest ML models, can deliver meaningful and interpretable predictions.
- Real-world context matters: Choosing a passing threshold of 40 marks makes the output practically meaningful, not just mathematically correct.
- Machine learning is not magic: The algorithm is a mathematical procedure that can be understood, implemented, and debugged just like any other code.

5.3 Areas Identified for Future Improvement

Through building this project, several directions for future development were identified:

- Incorporate multiple features (e.g., attendance, sleep hours, participation) using Multiple Linear Regression.
- Use a larger, real-world dataset sourced from open repositories like Kaggle or UCI ML Repository.
- Evaluate model performance using metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared (R^2).
- Build a graphical user interface (GUI) using Tkinter or a web interface using Flask.
- Explore classification-first approaches (e.g., Logistic Regression, Decision Trees) rather than regressing and then thresholding.

6. Project Architecture and Code Overview

6.1 Folder Structure

```
Student-performance-predictor-main/   ├── data/   │   └── data.csv
← Training dataset   ├── Source code/   │   ├── model.py            ← ML
model + data loading   │   └── main.py              ← User interface +
prediction   └── README.md               ← Project documentation
```

6.2 Component Descriptions

Component	Description
`LinearRegressionModel class`	Implements train() and predict() methods. Stores slope (m) and intercept (b) as instance attributes.
`load_data() function`	Opens data.csv using DictReader, parses study_hours and marks columns into Python lists.
`main() function`	Orchestrates the full flow: loads data, trains model, accepts user input, prints predicted marks and PASS/FAIL result.
`data.csv`	9-row dataset with study_hours, attendance, and marks columns. Acts as the training corpus for the model.

6.3 System Flow

The end-to-end execution flow of the project is as follows:

1. User runs main.py from the Source code directory.
2. load_data() reads data.csv and returns two lists: study_hours and marks.
3. model.train() computes the slope and intercept using the OLS formulae.
4. The user enters the number of study hours.
5. model.predict() applies y = mx + c to produce a predicted marks value.
6. The result is classified as PASS (marks ≥ 40) or FAIL (marks < 40) and printed to the console.

7. Conclusion

This project successfully demonstrates the end-to-end pipeline of a simple machine learning application — from raw data to a working prediction system. Despite being a beginner-level project, it encompasses several fundamental concepts that form the backbone of applied machine learning: data ingestion, supervised learning, mathematical optimization, prediction, and result interpretation.

What makes this project particularly valuable as a learning exercise is the decision to implement the algorithm from scratch. By manually coding the slope and intercept calculations rather than using library functions, the developer gains a deep, hands-on understanding of how linear regression actually works — not just how to call it.

The project also reflects good software engineering practices: a modular codebase, separation of concerns, external data files, error handling, and version control. These habits, established early, will serve as a strong foundation for more complex future projects.

Looking forward, there are clear, well-identified paths to extend this work: richer datasets, additional features, better evaluation metrics, and more sophisticated models. But as a first step into the world of AI and machine learning, this project achieves exactly what it sets out to do — it works, it teaches, and it inspires further exploration.